Senior Design - Intermediate Planning

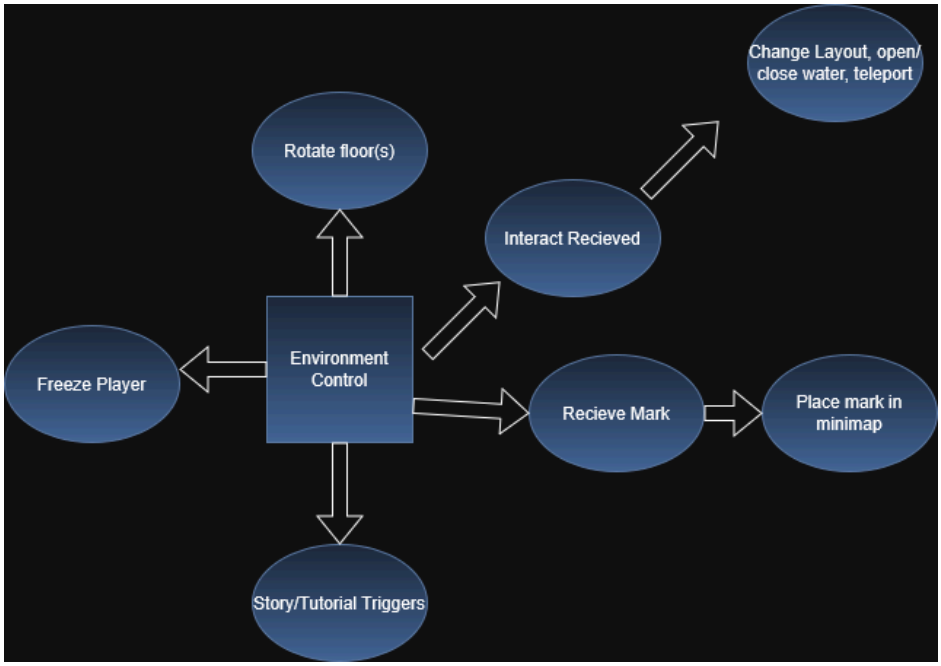
1. Minimum viable project: Our minimal viable project is a simple, tiered tower that the user can play through. Each of the tiers of the tower will be able to rotate. This rotation is the core of the gameplay, changing the orientation of the player, the environment, and puzzles to create new gameplay states. When rotation is happening, there will be a short shaking camera animation that will play, signaling the player that the tower is rotating, which will keep them in place so as to not make the physics more complicated than they need to be. There will be ways to rotate the tower within the tower itself, as well as rotating the tower from a map that will allow the user to keep track of the orientation of the different levels.

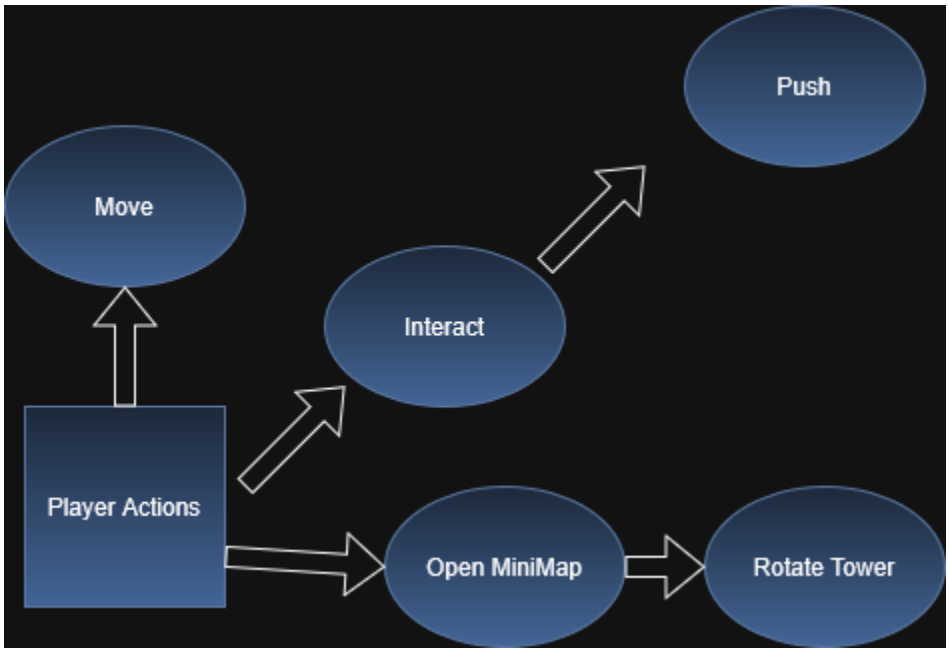
   This game will primarily be a puzzle game, which can be solved by finding different ways to move through the tower, interact with things in the environment, and tower rotation. We will have circuit puzzles, combination puzzles, water puzzles (involves water physics with ray casting), and Hades puzzles, which involves the character "dying" and being able to use bridges that they were unable to use before. The combination of these puzzle options combined with the rotation aspect is what makes this game unique and the puzzles within interesting.

   To make our game more interesting and have more depth, we will also incorporate story elements. This includes a plot and voice acting that will encourage the player through the environment. The story of our game helps in that the user will gain more insight as they play along, as there will be voice acting and hints which will guide the user as they go about the tower. The user will have an inventory in order to carry objects to solve puzzles, as well as a mini map which will aid them in seeing how the tower rotates and how it affects the environment around them.

2. Descriptions and diagrams of our system, sub-systems/components, and expected user interactions:

   Environment Diagram and Player Diagram





3. Specific tasks (and dependencies between tasks) that need to be completed in order to implement the systems and subsystems listed above:

Player system (all requirements/resources are all using Unity):

- Character movement:
  Basic player movement. This should be physics driven to best interact with the water system. Essentially adding force to a rigidbody to move in the direction associated with

the camera. This only handles basic WASD movement, which should be suitable up to the point of ladder/climbing to higher levels.
    - Constraints: No known constraints
    - Dependencies: No known dependencies
- Character Climbing:
  Basic vertical movement for the character. This will be done through interacting with ladders/climbable surfaces. This will deactivate the normal movement, use transforms to move the player to the next location, move the player off the climbable, then restore normal WASD movement.
    - Constraints: No known constraints
    - Dependencies: Basic Movement System and Rotation Mechanics.
- Minimap:
  The player will be able to press a button, i.e. TAB that pulls up a smaller version of the whole Tower from an outside perspective. This should contain markers for floor differentiation, the player position, and include the ability to transform some floor levels through the mini-map. This will also disable player movement and pause any ongoing tower rotation while the map is active. This will be tied to the movement of the full scale tower and keep a constant marker of the player on the map.
    - Constraints: No known constraints
    - Dependencies: Tower, tower movement, player movement, and surrounding environment
- Interaction System:
  The player will need to interact with the environment in three main ways: Grabbing/ placing items, placing markers in the environment, and rotating the environment. All of this will be handled by the same system. Grabbing things from the environment is a core part of the puzzle design of the game, so simply grabbing items to place in the inventory must be done properly. This will come in the form of a simple ‘E’ press following a prompt, allowing the player to add the item to their inventory. This behavior will be followed by using the inventory system to handle what can be done with a given item, whether it can be placed, its orientation, and its necessity for the puzzle. There will also be the ability to place some items in various locations to help the player track the rotation of the tower on the minimap.
    - Constraints: No known constraints
    - Dependencies: Inventory, Assets, Puzzle Areas, Interaction Points, Puzzle Design, Minimap
- Inventory:
  Simple bar at the bottom of the screen that fills up as items are collected. The items will be displayed using an Image asset, and when selected, can be placed in the environment using the interaction system or used based on their individual characteristics. This must

have an array of potential slots, the ability to display the images, swap between a selected item, and show different UI elements to display what can be done with them.

- Constraints: No known constraints
- Dependencies: Assets, Interaction System

Environment system:

- Assets:
  This refers to a very broad set of 3D models that will be designed from scratch for the project. All of the models must have a cohesive look, be designed to work well together, and be minimal poly-count for optimization. Environmental Assets, Puzzle Assets, UI assets, and Interactable assets are the main categories to be done. Everything in these categories must be completed and textured using Substance Painter.
  - Constraints: No known constraints
  - Dependencies: No known dependencies
  - Requirements/resources: Blender/Substance Painter
- Animation:
  There will be some animation that must be done to handle various interactions in game, these will mostly be used for particle effects when items are placed, the tower is rotated, and puzzle visual effects. Such as water splashing, electricity arcing, and smoky shadows.
  - Constraints: No known constraints
  - Dependencies: Need to have assets completed first
  - Requirements/Resources: Blender/Unity particle system
- Puzzles:
  This refers to basic puzzle design. Puzzles must be designed to fit the tower shape, and all potential states for the puzzle must be known. Then implementing the puzzles into the environment with scripting and asset placing.
  - Constraints: Unity
  - Dependencies: Assets/story
  - Requirements/resources: No known requirements/resources
- Tower movement:
  The tower movement will need to be handled from multiple states, both when the player triggers rotation, and when a certain stage of the puzzle is reached. This will be done through scripting a simple transform.rotation script. By handling it with a simple script, we can branch out and use it to create larger and larger structures to scale up once our Minimum Viable Product is met. This rotation must freeze player movement, rotate one or more levels of the tower at once, and be set up in such a way that the puzzles move properly with the tower rotation.
  - Constraints: No known constraints
  - Dependencies: Tower assets, Player movement
  - Requirements/resources: No known requirements/resources

- Voice/caption implementation:
  There will be a lot of voice in this game, to add most of the story aspect to the game as well as provide advice and help the player complete the game. This will require the recording of voice lines, setting them to trigger and play at certain points during the gameplay, and deciding if they will finish fully or if they can be interrupted. A caption system will display the words that are spoken when they come up, indicate the end of a spoken line, and be able to move to the next line of text early (potentially)
    - Constraints: No known constraints
    - Dependencies: Game progress triggers, story, voice lines, caption system
    - Requirements/resources: UW recording studio/editing software, audacity
- Hint implementation:
  Once puzzles are determined, writing and recording hints using the captions/voice and visuals to help the user progress through the puzzle. These hints must be able to be helpful while not giving away the puzzle and refraining from being too obnoxious. Much playtesting required.
    - Constraints: No known constraints
    - Dependencies: Puzzle Design, Map Design, Story, Assets, full (at least prototyped) system
    - Requirements/resources: No known requirements/resources
- Story:
  Every good game has a good story backing it up. Must create a story that will be the guiding force for the game. This should engage the player, provide good motivation for their actions, and be able to be displayed with only visuals and some spoken lines.
    - Constraints: No known constraints
    - Dependencies: No known dependencies
    - Requirements/resources: No known requirements/resources
- Map:
  The map itself must be designed, this should be done based on the puzzle design and done in separate levels. Each tier of the tower needs to be designed on its own while still providing a fully cohesive whole. This involves placement of features, knowing how each tier can rotate, and the ability to interact with the other tiers.
    - Constraints: Potential learning curves with designing a fully functional map
    - Dependencies: No known dependencies
    - Requirements/resources: Blender/Substance Painter
- Render texture upscaling implementation:
  The visual style of this game will be similar to that of the game Devil Daggers, this means that each asset will have fully scaled textures that will be reduced in scale and then projected over the model to give the appearance of a lower visual fidelity while preserving model quality.
    - Constraints: Learning curve

    - Dependencies: No known dependencies
    - Requirements/resources: Done in Unity by changing the resolution of textures created in Substance Painter
- Color design:
  Coming up with a constant visual feel of the game will require selecting a good color pallet. This will help with the creation of physical designs and create a consistent mood for everything in the game to avoid the clashing visual feel that some games can create.
    - Constraints: No known constraints
    - Dependencies: No known dependencies
    - Requirements/resources: No known requirements/resources